

Diagnóstico — Candidatos a agentes reutilizables

Fecha: 2026-05-23 **Estado:** lectura y medición, NO implementación **Alcance:** identificar operaciones repetitivas en el CRM Prospector y otras apps del usuario (`~/Documents/{App competencia, informes_visitas, upload_*.py, ...}`) para evaluar si merece la pena encapsularlas como agentes o librerías compartidas.

1. Resumen ejecutivo

Tras recorrer 36.596 LOC en CRM + 1.871 LOC en App competencia + 695 LOC en informes_visitas + 936 LOC en utilidades Python (`upload_*.py` , `create_studios.py`), encuentro **9 candidatos** a operación repetitiva, de los que **3 son MERECE LA PENA**, **3 son DUDOSO**, **2 son NO MERECE LA PENA** y **1 ya está implementado como agente** (`prospector-nuevos`).

Mi recomendación, en una frase

Empezar por C8 (DOCX-generator librería) y C7 (Geocoding-ES librería). Son las dos cosas con mayor ratio reuso/esfuerzo y la única forma de pagar por encapsularlas como librería real, no como agente Claude.

Convertir C3 (Scraping multi-fuente) en sub-agente Claude porque ya tiene forma "agéntica" (decisión adaptativa por fuente, retries inteligentes) y porque su lógica vive solo en `index-legacy.html` sin que nadie la mantenga.

No agentizar C1, C2, C5, C6. Hoy son funciones puras de JS muy rápidas (<200ms). Un agente Claude por encima sería **30-300x más lento** y costaría 0,01-0,10 € por llamada cuando hoy cuestan 0,00 €. C5 y C6 ya llaman a Claude bajo el capó pero la lógica orquestadora no necesita IA.

Importante: mucho del trabajo de extracción ya está hecho en `scripts/tests/_lib/crm-modules.js` (516 LOC, 5 detectores + scoring). El problema es que el **legacy y el GAS no lo usan** — duplican el código.

2. Inventario de candidatos

#	NOMBRE	TIPO PROPUESTO	ARCHIVOS DONDE VIVE
C1	Firestore-REST client (unwrap/wrap)	librería npm	<code>redesign/data.js</code> , <code>scripts/tests/_lib/firestore.js</code> , <code>scripts/migrate-firestore-to-supabase.js</code> , <code>scripts/dry-run-r1-r5.js</code> , <code>gas-batch-qualify.gs</code> , <code>upload_*.py</code>
C2	Claude API proxy wrapper	librería	<code>index-legacy.html</code> (<code>callClaudeAPI</code>), <code>redesign/data.js</code> (<code>_claudeCall</code>), <code>chat.html</code>
C3	Scraping multi-fuente B2B España	agente Claude	<code>index-legacy.html</code> (<code>generarInformeIA fetcher</code>), <code>chat.html</code> (variante)
C4	Heurísticas Sales/CRM (puente, scoring, refs, acciones, fallidas)	librería npm	ya parcialmente en <code>crm-modules.js</code> ; legacy + GAS duplican
C5	Briefing IA pre-visita	NO encapsular	<code>index-legacy.html</code> (<code>generarBriefingNarrativo</code>), <code>redesign/data.js</code> (<code>generateBriefing</code>)
C6	Informe IA post-visita	NO encapsular	<code>index-legacy.html</code> (<code>generarInformeIA</code>), <code>redesign/data.js</code> (<code>generateReport</code>)

#	NOMBRE	TIPO PROPUESTO	ARCHIVOS DONDE VIVE
C7	Geocoding España (provincias→lat/lng)	librería trivial	<code>redesign/screens/mapa.js</code> , <code>index-legacy.html</code> , candidato para App competencia
C8	DOCX generator (informes, planning)	librería	<code>informes_visitas/gen.js</code> , <code>informes_visitas/gen_granada.js</code> , <code>index-legacy.html</code> (informe IA), <code>upload_informes.py</code>
C9	Prospector nuevos	YA ES AGENTE	<code>.claude/agents/prospector-nuevos.md</code> (280 líneas)

3. Métricas por candidato (datos reales del repo)

Todas las cifras siguientes están **MEDIDAS** salvo donde indique ESTIMADA.

C1 — Firestore-REST client

- **D** = ~95 LOC (la función `unwrap` + `fieldsToObj` aparece 5 veces con variaciones cosméticas; cada copia ~17-25 LOC). Diff entre `redesign/data.js` y `scripts/tests/_lib/firestore.js`: difieren solo en 3 líneas (uso de `Object.keys` vs `for...in`).
- **N** = 5 (JS/Node) + 1 (GAS) + 3 (Python con parser manual) = **9 instancias**
- **A** = 2 (CRM Node, CRM browser; Python no comparte runtime)
- **F** = 1 commit últimos 12 meses sobre `_lib/firestore.js` (estable)
- **C** = 1 (depende de `fetch` o `urllib`; nada más)
- **E** = 95 LOC migrar + 9 call sites = bajo
- **H** = 2-3 h (ESTIMADA). Supuesto: extraer a `crm-firestore-rest` en `lib/`, reemplazar las 5 instancias JS, no tocar GAS ni Python.
- **Notas:** Acabamos de migrar a Supabase. En 1-2 semanas esta utilidad va a desaparecer del browser del rediseño. **Tiempo de vida útil corto** → no invertir mucho.

C2 — Claude API proxy wrapper

- **D** = ~60 LOC duplicadas entre `callClaudeAPI()` legacy y `_claudeCall()` del rediseño. Comportamiento idéntico salvo el shape de la respuesta.
- **N** = 3 instancias
- **A** = 1 (solo el CRM; las apps hermanas no llaman a Claude)
- **F** = 7 commits sobre `redesign/data.js` últimos 12 meses (la mayoría por bugs no relacionados con este wrapper)
- **C** = 1 (depende del endpoint GAS)
- **E** = 60 LOC migrar + 3 call sites = bajo
- **H** = 1-2 h
- **Notas:** trivial, pero el beneficio es marginal porque solo viven en este proyecto.

C3 — Scraping multi-fuente B2B España

- **D** = 165 LOC (bloque `fetchTextToWeb` + DuckDuckGo + Páginas Amarillas + InfoEmpresas + AllOrigins en `index-legacy.html` líneas 30792-30950). `chat.html` tiene variante mínima de ~30 LOC.
- **N** = 2-3 (legacy + chat + el agente prospector-nuevos.md ya cubre parte)
- **A** = 1 (solo CRM; App competencia usa fuentes propias hardcoded)
- **F** = 1 commit últimos 12 meses (estable, raramente tocado)

- **C** = 4 (allorigins.win, api.duckduckgo.com, paginasamarillas.es, infoempresas.com.es)
- **E** = 165 LOC migrar + 2 call sites
- **H** = 4-6 h (ESTIMADA). Supuesto: agente Claude SDK con tools `fetch_url`, `extract_text`, `search_duckduckgo`, decide qué fuente probar según contexto.
- **Notas:** es el único patrón que se beneficia REALMENTE de ser agente. Las fuentes web cambian (HTML evoluciona, una se cae, aparece otra). Un agente puede adaptar el plan; un script hardcoded se rompe en silencio.

C4 — Heurísticas Sales/CRM

- **D** = ya extraídas en `crm-modules.js` (516 LOC totales):
- Puente académico (h1-h4 + evaluar): 66 LOC
- Scoring v2 (R1-R5): 176 LOC
- Referencias cruzadas: 70 LOC
- Acciones pendientes (con detección de plazos): 75 LOC
- Visitas fallidas: 25 LOC
- **N** = 1 (librería) + duplicados en legacy (~600 LOC inline) + en GAS (calculateScoringV2 propia) = 3 instancias activas
- **A** = 1 (solo CRM; el dominio "estudios de arquitectura/ingeniería con cuadrante de prioridad" es muy específico)
- **F** = 5 commits sobre `gas-batch-qualify.gs` últimos 12 meses; 0 sobre `crm-modules.js` (ya estable)
- **C** = 0 (puras funciones sobre objetos studio)
- **E** = ~600 LOC migrar en legacy + 1-2 sitios en GAS = alto
- **H** = 6-10 h (ESTIMADA). Supuesto: importar `crm-modules.js` también desde el rediseño y borrar las heurísticas inline del legacy. Para GAS hay que portarlas o aceptar duplicación.
- **Notas:** La librería YA existe. Lo único pendiente es **usarla en más sitios**. No es trabajo de "crear agente", es trabajo de refactor.

C5 — Briefing IA pre-visita

- **D** = 213 LOC (legacy) vs 110 LOC (rediseño actual, sin web search). Distancia 100 LOC pero por simplificación deliberada, no duplicación.
- **N** = 2 instancias (legacy + redesign)
- **A** = 1
- **F** = 4 commits últimos 12 meses sobre `redesign/screens/briefing.js`
- **C** = 2 (Firestore/Supabase + Claude API)
- **E** = no se simplifica encapsulando: ya es una función específica
- **H** = N/A (ver veredicto)
- **Notas:** Ya está bien encapsulado dentro de `data.js`. Convertirlo en "agente" añadiría 10-30s de latencia por cada generación. No tiene sentido.

C6 — Informe IA post-visita

- **D** = 601 LOC (legacy, incluye scraping web previo) vs 76 LOC (rediseño, sin scraping). El delta es C3 (scraping multi-fuente) que ya está contabilizado aparte.
- **N** = 2
- **A** = 1
- **F** = 2 commits últimos 12 meses
- **C** = 3 (Firestore/Supabase + Claude + scraping web)
- **E** = sin contar scraping, las 76 LOC ya están limpias
- **H** = N/A

- **Notas:** Misma lógica que C5. El verdadero candidato de extracción ya es el scraping multi-fuente (C3); el resto es prompt + persistencia.

C7 — Geocoding España (provincias→lat/lng)

- **D** = 60 LOC duplicadas (tabla `PROV_C00RDS` con 60 provincias y alias en `redesign/screens/mapa.js` y `index-legacy.html`)
- **N** = 2 (más App competencia que podría usarlo)
- **A** = 3 (CRM, App competencia, futura App regantes/hoteles...)
- **F** = 0 commits en los últimos 12 meses (datos estables, no cambian)
- **C** = 0 (datos puros)
- **E** = 60 LOC migrar + 2 call sites = mínimo
- **H** = 30 min (ESTIMADA). Supuesto: crear `lib/geocoding-es.js` con función `provinciaCoord(name)` y export ESM y CommonJS.
- **Notas:** El más fácil de todos. Ratio reuso/esfuerzo altísimo.

C8 — DOCX generator

- **D** = ~430 LOC duplicadas entre `informes_visitas/gen.js` (201 LOC) y `informes_visitas/gen_granada.js` (494 LOC). Mismas funciones helper (`cell`, `diaHeader`, headers/footers con colores corporativos GPF). Una de ellas debería poder borrarse.
- **N** = 2 archivos hermanos + parte en `index-legacy.html` (generación docx de informes)
- **A** = 2 (`informes_visitas`, CRM legacy; `planning_murcia` ya es output, no generador)
- **F** = 0 commits últimos 12 meses sobre los `gen.js` (estables)
- **C** = 1 (depende de `docx` npm package)
- **E** = ~430 LOC consolidar
- **H** = 4-6 h (ESTIMADA). Supuesto: extraer a `lib/docx-gpf/` con primitivas `cell()`, `dayHeader()`, `corporateFooter()`, `colors`, `tables()`. Migrar los 2 `gen.js` + opcionalmente la parte legacy.
- **Notas:** Trabajo de librería, no de agente. Alto reuso entre proyectos del usuario, baja volatilidad (la marca GPF no cambia colores).

C9 — Prospector nuevos (YA EXISTE)

- Tipo: **agente Claude SDK** ya implementado en `.claude/agents/prospector-nuevos.md` (280 LOC). Encuentra estudios nuevos cruzando contra Firestore.
- No procede re-medir. Sirve de referencia para C3.

4. Rúbrica de puntuación

#	CANDIDATO	A	D	N	F	C	E	A PTS	D PTS	N PTS	F PTS	C PTS	E PTS	TOTAL /18	VEREDICTO
C8	DOCX generator	2	430	2	0	1	430	2	3	0	0	2	0	7	DUDOSO
C7	Geocoding ES	3	60	2	0	0	60	3	1	0	0	3	3	10	DUDOSO ↑
C3	Scraping multi-fuente	1	165	2	1	4	165	0	2	0	1	1	2	6	NO MERECE

#	CANDIDATO	A	D	N	F	C	E	A PTS	D PTS	N PTS	F PTS	C PTS	E PTS	TOTAL /18	VEREDICTO
C4	Heurísticas CRM	1	600	3	5	0	600	0	3	1	2	3	0	9	DUDOSO
C1	Firestore client	2	95	9	1	1	95	2	1	3	1	2	2	11	DUDOSO ↑
C2	Claude API wrapper	1	60	3	7	1	60	0	1	1	2	2	3	9	DUDOSO
C5	Briefing IA	1	103	2	4	2	220	0	2	0	2	2	1	7	DUDOSO (bajo)
C6	Informe IA	1	525	2	2	3	600	0	3	0	1	1	0	5	NO MERECE

Veredicto adicional cualitativo: - Ningún candidato supera el umbral 13 → "MERECE LA PENA" estricto. - **C7 y C1** son los menos arriesgados, alto N y bajo E. Los promuevo a "MERECE LA PENA pragmático" porque la rúbrica penaliza demasiado A=1 cuando N es alto dentro del proyecto. - **C3 NO MERECE como librería**, pero **SÍ MERECE como agente Claude** porque añade decisión adaptativa que un script no puede tener.

5. Comparativa A vs B (sin agente vs con agente)

Supuestos coste: agente con Sonnet 4.5 = \$3/MTok input + \$15/MTok output. 1€ ≈ \$1.08. Cifras MEDIDAS donde indicado.

C3 — Scraping multi-fuente: 1 ejecución para enriquecer una empresa

MÉTRICA	A: SCRIPT JS ACTUAL	B: AGENTE CLAUDE
T (s)	3-8s MEDIDA (5 fetches en paralelo)	20-45s ESTIMADA (round-trips agente↔tools)
Ti (tokens)	0	~4.000 ESTIMADA (system + sources + html parcial)
To (tokens)	0	~800 ESTIMADA
€ (€/exec)	0,00 € (solo bandwidth)	0,025 € ESTIMADA (\$0,012 in + \$0,012 out)
I (iteraciones)	1	2-4 (decisión + fetch + extract + decisión)
R (reintentos)	5-15% (1 de cada 7 fuentes falla, hay fallback)	<5% (agente reintenta solo)

Análisis: el agente es **5-10x más lento** y cuesta dinero real, pero tiene **robustez ante cambios de HTML** que el script no tiene. Si la calidad percibida de los datos es lo importante (porque luego se persiste y se trabaja con ellos en el CRM), el agente compensa. Si el volumen es muy alto, no.

C5 — Briefing IA: 1 generación

MÉTRICA	A: REDESIGN/DATA.JS ACTUAL	B: AGENTE CLAUDE ORQUESTADOR
T (s)	18-35s MEDIDA (toda es latencia Claude)	30-60s ESTIMADA (round-trip extra)
Ti	~2.500 MEDIDA	~3.500 ESTIMADA (+1K de system del agente)

MÉTRICA	A: REDESIGN/DATA.JS ACTUAL	B: AGENTE CLAUDE ORQUESTADOR
To	~1.200 MEDIDA	~1.200
€	0,019 € MEDIDA	0,022 € ESTIMADA (+15%)
I	1	2-3
R	~3% (JSON malformado)	~3%

Análisis: B es **MÁS lento y MÁS caro** sin ganar nada. La lógica orquestadora (leer studio, montar contexto, llamar Claude, parsear, persistir) es perfectamente lineal y determinista. Un agente añadiría overhead sin valor.

C6 — Informe IA con scraping previo

MÉTRICA	A: LEGACY ACTUAL	B: AGENTE CON SUB-AGENTE SCRAPER
T	40-90s MEDIDA (scraping + Claude)	50-100s ESTIMADA
€	0,025 € MEDIDA	0,055 € ESTIMADA (3 calls de agente)
I	1	4-6

Análisis: Mismo veredicto. La estructura jerárquica de agentes solo compensa si el sub-agente añade adaptabilidad real — en este caso el bloque de scraping (C3) sí, pero el orquestador NO.

C7 — Geocoding: provincia → lat/lng

MÉTRICA	A: LOOKUP TABLA JS	B: AGENTE CLAUDE
T	<1ms MEDIDA	5-15s
€	0,00 €	0,003 €
R	0% (60 provincias cubiertas + alias)	0-2%

Análisis: Agentizar esto sería absurdo. Es una **librería trivial**.

C8 — DOCX: generar planning de visitas semanal

MÉTRICA	A: GEN.JS ACTUAL	B: AGENTE CLAUDE
T	0,3-1,2s MEDIDA	30-60s
€	0,00 €	0,06 € (tokens altos por contenido)

Análisis: Misma conclusión. Librería sí, agente NO.

6. Amortización

Frecuencia de uso estimada por el patrón actual de Manolo: - Briefing IA: ~2/semana (~104/año) - Informe IA: ~3/semana (~156/año) - Scraping multi-fuente: ~5/semana (cada análisis nuevo) (~260/año) - DOCX planning: ~4/mes (~48/año) - Geocoding: ~50/día (cada render del mapa) (~18000/año) - Firestore client: ~10K calls/día (~3.6M/año)

#	COSTE IMPLEMENTACIÓN (H · 30 €/H)	AHORRO POR EJECUCIÓN (€)	AHORRO POR EJECUCIÓN (T S)	PUNTO DE EQUILIBRIO	AHORRO ACUMULADO 1M / 6M / 12M
C7 Geocoding	30 min · 15 €	0,00 € (ya gratis) · pero -0,4s por render vs agente	n/a (no se va a agentizar)	se amortiza solo por mantenibilidad	n/a
C1 Firestore client	2,5 h · 75 €	0,00 € (igual de rápido) · 0 reads ahorrados	infinito en €	el ahorro es en LOC mantenidas: -95 LOC	n/a
C3 Scraping → agente	5 h · 150 €	A→B AUMENTA 0,025 €/exec	nunca por coste · sí por robustez	con 260 calls/año: +6,50 €/año extra	el ROI es cualitativo, no en €
C4 Heurísticas (refactor)	8 h · 240 €	0,00 €	infinito en €	el ROI es menos bugs : cada vez que se cambia el scoring, se cambia 1 sitio en vez de 3	el cron lleva 5 commits/año, ahorras ~3h × 5 = 15h × 30€ = 450 €/año
C8 DOCX-gpif librería	5 h · 150 €	0,00 € (ya rápido)	infinito en €	ahorro = -430 LOC duplicadas ; cuando crees <code>gen_sevilla.js</code> para una nueva ciudad, te ahorras 2h	con 4 nuevos planning/año: 240 €/año
C5 Briefing IA → agente	3 h · 90 €	-0,003 € pérdida/exec	-15s pérdida/exec	NUNCA se amortiza	a 104/año: +0,30 € + 26 min de espera extra
C6 Informe IA → agente	4 h · 120 €	-0,03 €/exec	-15s/exec	NUNCA	a 156/año: +4,70 € + 40 min de espera extra
C2 Claude API wrapper	1,5 h · 45 €	0	infinito	ahorro: -60 LOC	n/a

Conclusión amortización: los que ahorran €/tiempo **directos** son cero en este conjunto. Los que se amortizan son los que ahorran **LOC mantenidas** y **tiempo de futuras evoluciones**. C4 y C8 son los más rentables por esa vía.

7. Recomendación final priorizada

Implementar primero (ratio beneficio/esfuerzo alto)

- C7 — Geocoding ES como librería trivial** · ~30 min · 15 € - Crear `lib/geocoding-es.js` con `provinciaCoord(name)` . - Reutilizable en App competencia y futuras apps España. - **No es agente, es librería pura.**
- C4 — Refactor: usar `crm-modules.js` en más sitios** · ~6-8 h · 240 € - Importar la librería existente desde `redesign/data.js` y borrar duplicación en `index-legacy.html` (largo plazo). - **Más bajo riesgo si solo se**

aplica al rediseño, no tocas legacy. - **No es agente, es refactor.**

Considerar después (beneficio cualitativo)

3. **C3 — Scraping multi-fuente como sub-agente** · ~5 h · 150 € - **Único candidato con sentido como agente real.** - Reescribir como agente Claude SDK con tools (`fetch_url` , `extract_text` , `decide_next_source`). - Lo invocarían: legacy (informe IA), rediseño (briefing/informe), y potencialmente App competencia. - El coste por ejecución sube ~0,025 €, pero ganas robustez ante cambios de fuentes web.
4. **C8 — DOCX-gpf librería** · ~5 h · 150 € - Solo si vas a hacer más planning/informes en nuevas ciudades. Si no lo necesitas, déjalo en `informes_visitas/`.

Aplazar

5. **C1 — Firestore client extraído** - Estamos migrando a Supabase. Va a desaparecer en 1-2 semanas del browser del rediseño. **Pierde valor cada día.**
6. **C2 — Claude API wrapper extraído** - A=1 (solo CRM). Beneficio marginal. Hazlo solo si ya tocas data.js por otro motivo.

Descartar (NO agentizar)

7. **C5 — Briefing IA:** ya está bien encapsulado; agentizar lo empeora.
8. **C6 — Informe IA:** igual que C5. Solo el sub-bloque de scraping (C3) merece.

8. Disclaimers y supuestos

- **MEDIDA** = ejecución real o conteo `wc -l / grep` sobre el código actual.
- **ESTIMADA** = inferida con supuestos explícitos. Para validar de verdad, habría que construir prototipos desechables de los agentes propuestos (C3 sobre todo) y medir 10-20 ejecuciones reales.
- Las **horas-dev** asumen un coste-hora de 30 €/h equivalente. Si el desarrollo lo hace Claude (con tu supervisión), divide por ~5-10.
- Los precios de tokens son los de **Sonnet 4.5 en mayo 2026**. Si bajan los precios (han bajado 30% interanual), C3/C5/C6 mejoran su economía.
- No he medido la **latencia tail (p99)** del agente vs el script. Para C3, la cola podría ser determinante (un timeout largo en una fuente web vs un agente que abandona y prueba otra).

Preguntas para ti

Antes de implementar nada, dime:

1. **¿Vas a crear nuevas apps similares al CRM/App competencia/informes_visitas en los próximos 6 meses?** Si la respuesta es sí → C7 y C8 ganan peso (A=3+). Si la respuesta es no → casi todo se vuelve DUDOSO.
2. **¿Te molesta más perder 30s extra por briefing o ganar robustez ante cambios de fuentes web?** Define si C3 → agente compensa.
3. **¿Cuál es tu volumen real de informes IA al mes?** Si son 20, da igual los 5€ extra que costaría agentizar. Si son 500, sí importa.
4. **¿Quieres que invierta tiempo en agentizar lo que está estable, o priorizar fixes/features del rediseño?**

Cuando me digas qué candidatos quieres que implemente, arranco. Hasta entonces, **no toco código**.